BLOQUE II - TEMA 3

TECNOLOGÍA BÁSICA

Tipos abstractos y Estructuras de datos. Organizaciones de ficheros. Algoritmos.

Formatos de información y ficheros.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. TIPOS ABSTRACTOS DE DATOS (TAD) 5

2. ESTRUCTURAS DE DATOS 7

2.1 Clasificación general 7

2.2 Estructuras lineales 8

2.2.1 Listas 8

2.2.2 Pilas (Stacks) 10

2.2.3 Colas (Queues) 11

2.3 Estructuras no lineales 12

2.3.1 Árboles 12

2.3.2 Grafos 14

2.4 Estructuras de acceso directo 16

2.4.1 Tablas hash 16

2.4.2 Diccionarios 17

2.5 Conjuntos (Sets). 18

3. ALGORITMOS 19

3.1 Tipos de algoritmos 19

3.1.1 Algoritmos de búsqueda 19

3.1.2 Algoritmos de ordenación 21

3.2 Recursividad 22

3.3 Algoritmos sobre estructuras de datos 23

3.3.1 Inserción y eliminación 23

3.3.2 Recorrido de árboles y grafos 24

4. ORGANIZACIONES DE FICHEROS 26

4.1 Modos de acceso a ficheros 26

4.1.1 Acceso secuencial 26

4.1.2 Acceso directo o aleatorio (random access) 27

4.1.3 Acceso indexado 27

4.2 Métodos de organización 28

4.2.1 Organización secuencial 28

4.2.2 Organización secuencial indexada 29

4.2.3 Organización directa o aleatoria 30

4.2.4 Organización por dispersión (hashing) 31

5. FORMATOS DE INFORMACIÓN Y FICHEROS 32

5.1 Formatos de texto 32

5.2 Formatos binarios 32

5.3 Compresión de ficheros 33

5.4 Formatos de ficheros comunes en bases de datos 33

5.5 Formatos multimedia y documentos 34

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

Supera. by thePower

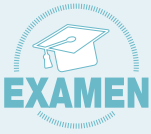
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. TIPOS ABSTRACTOS DE DATOS (TAD)

Un **Tipo Abstracto de Datos (TAD)** es un modelo matemático que describe un conjunto de datos y las operaciones válidas sobre ellos, **independientemente de su implementación concreta en un lenguaje de programación o en una máquina**.

Se llama "abstracto" porque define **qué operaciones se pueden realizar** sobre los datos y **qué comportamiento se espera** de ellas, sin entrar en los detalles de cómo se llevan a cabo.

Características principales de un TAD:



- **Encapsulamiento**: los datos y las operaciones están agrupados, y se oculta su implementación interna.
- **Interfaz definida**: se especifican claramente las operaciones permitidas (inserción, borrado, búsqueda, etc.).
- **Independencia de implementación**: un mismo TAD puede implementarse de formas distintas (con arrays, listas, punteros...).
- **Consistencia lógica**: el comportamiento esperado de las operaciones debe mantenerse, sin importar cómo estén implementadas.

Ventajas de un TAD:

El uso de tipos abstractos de datos ofrece varias ventajas en el desarrollo de programas:

- **Modularidad**: permite dividir el programa en componentes independientes y reutilizables.
- **Mantenibilidad**: los cambios en la implementación de un TAD no afectan al resto del programa si se mantiene la interfaz.
- **Reutilización**: un mismo TAD puede utilizarse en múltiples programas o contextos.
- **Legibilidad y claridad**: al centrarse en la funcionalidad, mejora la comprensión del código.
- Facilita la verificación: se pueden comprobar y validar las operaciones sin necesidad de conocer los detalles internos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Existen múltiples tipos abstractos de datos, algunos de los más utilizados en informática, y se verán más en profundidad en adelante, son:

- **Lista**: colección ordenada de elementos. Puede ser simple, doble, circular...
- **Pila (Stack)**: estructura LIFO (Last In, First Out). Se insertan y eliminan elementos por un solo extremo (top).
- **Cola (Queue)**: estructura FIFO (First In, First Out). Los elementos se insertan por un extremo y se eliminan por el otro.
- **Árbol**: estructura jerárquica de nodos. Ej.: árbol binario de búsqueda (BST), árbol AVL, etc.
- **Grafo**: conjunto de nodos conectados mediante aristas. Puede ser dirigido o no, ponderado o no.
- **Conjunto (Set)**: colección de elementos sin duplicados.
- **Diccionario o tabla de símbolos**: estructura que almacena pares clave-valor.

El concepto de TAD separa la **interfaz** (qué operaciones se pueden hacer) de la **implementación** (cómo se hacen internamente).

Interfaz:

- Describe las operaciones que se pueden realizar (por ejemplo, insertar, eliminar, buscar).
- Define el comportamiento de dichas operaciones, sin especificar cómo se llevan a cabo.

Implementación:

- Puede variar según necesidades de eficiencia, estructura de datos utilizada (arrays, listas enlazadas, etc.), lenguaje, etc.
- Ejemplo: una pila puede implementarse con un array o con una lista enlazada, manteniendo la misma interfaz.

Abstracción:

- Permite trabajar con estructuras de datos complejas como si fueran "cajas negras".
- Favorece el desarrollo de software robusto y con menor acoplamiento.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. ESTRUCTURAS DE DATOS

2.1 CLASIFICACIÓN GENERAL



Una **estructura de datos** es una forma de organizar y almacenar datos en un ordenador para que puedan utilizarse de forma eficiente. Dependiendo de su forma y comportamiento, las estructuras de datos pueden clasificarse en:

- **Estructuras estáticas**: tienen un tamaño fijo determinado en tiempo de compilación (ej. arrays).
- **Estructuras dinámicas**: su tamaño puede cambiar durante la ejecución del programa (ej. listas enlazadas).
- **Estructuras lineales**: los elementos se organizan de forma secuencial (listas, pilas, colas).
- **Estructuras no lineales**: los elementos se organizan jerárquica o gráficamente (árboles, grafos).
- **Estructuras de acceso directo**: permiten localizar elementos rápidamente mediante claves (tablas hash, diccionarios).

Tipo	Estructura	Características principales	Ejemplos
Estáticas	Definidas en tamaño fijo al momento de su creación. No pueden cambiar de tamaño en tiempo de ejecución.	· Tamaño fijo · Acceso rápido por índice · Poco flexibles	· Arrays (vectores) · Matrices
Dinámicas	Pueden crecer o reducirse en tiempo de ejecución, según necesidades del programa.	· Tamaño variable · Uso eficiente de memoria · Acceso secuencial o estructurado	· Listas enlazadas (simples, dobles, circulares) · Pilas y colas dinámicas · Árboles · Grafos · Tablas hash con encadenamiento

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2 ESTRUCTURAS LINEALES

Las **estructuras de datos lineales** organizan los elementos en **una secuencia**, donde cada elemento tiene un único anterior y un único siguiente (salvo el primero y el último). Se utilizan cuando los datos deben procesarse en orden o gestionarse por orden de llegada o prioridad. Las más comunes son las **listas**, **pilas** y **colas**.

2.2.1 Listas

Una **lista** es una colección ordenada de elementos, donde cada uno ocupa una posición relativa. A diferencia de los arrays, las listas **sí pueden crecer dinámicamente.**

Las **listas enlazadas** son estructuras de datos dinámicas formadas por nodos que se conectan entre sí mediante punteros. Cada nodo contiene al menos **dos partes**: un campo de **datos** y un campo de **enlace** que señala al siguiente nodo de la lista. Según cómo se gestionen esos punteros, existen diferentes **tipos de listas enlazadas**:

- **Lista simple enlazada**: Es la forma más básica.
    - Cada nodo tiene:
        - Un **dato**.
        - Un **puntero al siguiente nodo** de la lista.
        - **El último nodo apunta a null o None, indicando el final de la lista.**
    - El recorrido solo se puede hacer en una dirección (de izquierda a derecha).
    - La inserción y eliminación son rápidas si se conoce el nodo anterior.

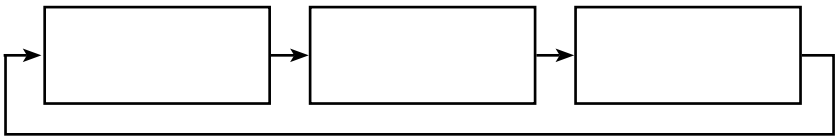


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

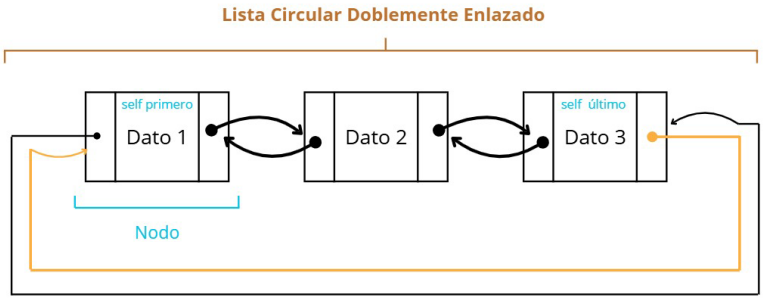
- **Lista doblemente enlazada.**
  - Cada nodo tiene:
    - Un **dato**.
    - Un **puntero al nodo anterior**.
    - Un **puntero al nodo siguiente**.
  - Se puede recorrer en ambos sentidos (adelante y atrás).
  - Facilita la eliminación de nodos intermedios sin recorrer toda la lista.
  - Requiere más memoria y lógica de gestión por tener dos punteros por nodo.



- **Lista circular**. En una lista circular, el último nodo no apunta a null, sino que se enlaza con el primero, formando un bucle cerrado. Puede ser:
  - **Simplemente enlazada circular**: cada nodo apunta solo al siguiente, y el último apunta al primero.



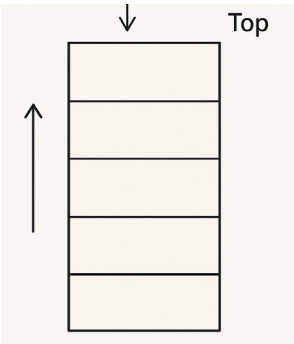
  - **Doblemente enlazada circular**: como la doble, pero también el último nodo apunta al primero, y el primero al último.
  - No tiene un final natural (no hay null).
  - Ideal para estructuras cíclicas, como listas rotativas o calendarios.
  - Puede complicar operaciones si no se gestiona bien el bucle.



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2.2 Pilas (Stacks)

Una **pila** es una estructura de datos lineal que funciona bajo el principio **LIFO (Last In, First Out)**, es decir, el **último elemento que se inserta es el primero que se extrae**. Las operaciones se realizan siempre en un único extremo, llamado **top**.



Tipos de pilas:

- **Pila estática:**
  - Se implementa mediante un array de tamaño fijo.
  - El tamaño debe conocerse de antemano.
  - Si se llena, puede producirse desbordamiento de pila (stack overflow).
- **Pila dinámica:**
  - Se implementa con una lista enlazada, donde cada nodo representa un elemento de la pila.
  - No tiene límite de tamaño (más allá de la memoria disponible).

Operaciones:

1. push(elemento): Inserta un nuevo elemento en el tope de la pila.
2. pop(): Elimina y devuelve el elemento que está en el tope.
3. peek() o top(): Devuelve el elemento en el tope sin eliminarlo.
4. isEmpty(): Devuelve True si la pila está vacía.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2.3 Colas (Queues)

Una **cola** es una estructura de datos lineal que funciona según el principio **FIFO (First In, First Out)**, lo que significa que el **primer elemento en entrar es el primero en salir**. Se utiliza cuando los datos deben procesarse en el **orden en que se generan**. Las operaciones se realizan en **dos extremos distintos**:

- En un extremo se **inserta** (enqueue).
- En el otro extremo se **elimina** (dequeue).



Existen distintas variantes de colas, según cómo se gestionen los extremos y el orden de extracción.

1. **Cola simple:**
   - Inserción por un extremo y eliminación por el otro.
   - Es la implementación más común.

2. **Cola circular:**
   - Los extremos se **unen formando un ciclo** lógico.
   - Muy útil cuando se usa un array fijo, ya que permite **reaprovechar posiciones vacías**.

3. **Cola con prioridad (Priority Queue):**
   - Cada elemento tiene una prioridad asociada.
   - Se extrae **el elemento con mayor prioridad**, independientemente del orden de entrada.
   - Puede implementarse con estructuras como **heaps** o **listas ordenadas**.

4. **Doble cola (Deque – Double Ended Queue):**
   - Permite tanto la **inserción como la eliminación por ambos extremos**.
   - Puede ser:
     - **Input-restricted**: inserción solo en un extremo, eliminación en ambos.
     - **Output-restricted**: eliminación solo en un extremo, inserción en ambos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Operaciones:

- **enqueue(elemento):** inserta un elemento al **final** de la cola.
- **dequeue()**: elimina y devuelve el **primer** elemento (el más antiguo).
- **peek():** devuelve el primer elemento sin eliminarlo.
- **isEmpty():** comprueba si la cola está vacía.
- **isFull():** útil en colas con tamaño fijo (como las circulares).

2.3 ESTRUCTURAS NO LINEALES

Las **estructuras no lineales** permiten organizar los datos en forma jerárquica o en redes complejas, no simplemente en una secuencia. En ellas, un elemento puede tener múltiples conexiones o relaciones con otros. Las más representativas son los **árboles** y los **grafos**.

2.3.1 Árboles

Un **árbol** es una **estructura de datos no lineal** que organiza los elementos de forma **jerárquica**. Está compuesto por **nodos conectados** mediante aristas, donde uno de ellos actúa como **raíz** y el resto se estructuran como **descendientes (hijos)**. Cada nodo puede tener:

- **Un único padre** (excepto la raíz).
- **Cero o más hijos.**



- **Altura de un nodo:** longitud del camino más largo hasta una hoja.
- **Profundidad de un nodo:** número de niveles desde la raíz hasta ese nodo.
- **Nodo hoja:** es aquel nodo de un árbol que no tiene ningún hijo (nodo terminal)

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipos de árboles:

- **Árbol binario:** Cada nodo puede tener como máximo dos hijos, denominados hijo izquierdo e hijo derecho.
- **Árbol binario de búsqueda (BST – Binary Search Tree).** Es un tipo específico de árbol binario con una propiedad clave, los **valores del subárbol izquierdo** de un nodo son **menores** que el valor del nodo, por lo tanto, los **valores del subárbol derecho** son **mayores**.
- **Árbol AVL:** Es un BST autoequilibrado, que mantiene una condición de balanceo, la diferencia de altura entre los subárboles izquierdo y derecho de cualquier nodo es como máximo 1:
- **Árbol rojo-negro (Red-Black Tree)**: Es otro tipo de BST autoequilibrado, en el que cada nodo tiene un color (rojo o negro) y se cumplen varias propiedades: la raíz es siempre negra, ningún nodo rojo puede tener un hijo rojo, y todo camino desde la raíz hasta un nodo hoja (nulo) contiene el mismo número de nodos negros. Estas reglas garantizan que la altura nunca sea mayor que el doble de la de un árbol perfectamente balanceado, ofreciendo O(log n) en inserción, búsqueda y borrado. Se usa en estructuras estándar de muchos lenguajes, como std::map/std::set en C++ o TreeMap/TreeSet en Java.
- **Árbol B:** Es un **árbol de búsqueda balanceado de orden m**, donde:
  - Cada nodo puede tener más de dos hijos (m hijos como máximo).
  - Todos los datos se almacenan en nodos internos y hojas.
  - Las hojas están siempre al **mismo nivel**.
- **Árbol B+:** Es una variación del árbol B, con diferencias clave:
  - Los datos solo se almacenan en las hojas.
  - Las hojas están **enlazadas secuencialmente** entre sí.
- **Árbol trie (árbol de prefijo):** Es un tipo especial de árbol utilizado para almacenar cadenas de texto donde:
  - Cada nodo representa un carácter.
  - Cada camino desde la raíz hasta un nodo marcado como fin de palabra representa una palabra completa (ese nodo no tiene por qué ser una hoja: si una palabra es prefijo de otra, como "sol" y "solo", "sol" termina en un nodo interno).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.3.2 Grafos

Un **grafo** es una estructura de datos no lineal compuesta por:

- Un conjunto de **nodos** (también llamados **vértices**).
- Un conjunto de **aristas** (o **arcos**) que conectan pares de nodos.

Los grafos permiten **modelar relaciones complejas entre entidades**, y son muy versátiles: pueden representar desde redes sociales hasta mapas de carreteras o flujos de trabajo.

Componentes de un grafo:

- **Vértice (nodo)**: elemento que representa una entidad (persona, ciudad, página web...).
- **Arista (arco)**: conexión entre dos vértices, que puede tener dirección y/o peso.
- **Grado de un vértice**: número de aristas que inciden en él.
- **Camino**: secuencia de vértices conectados por aristas.
- **Ciclo**: camino cerrado en el que se repite un vértice.
- **Subgrafo**: parte de un grafo que forma otro grafo por sí solo.

Tipos de grafos

- **Grafo dirigido (dígrafo)**
  - Las aristas tienen una dirección específica.
  - Se representan como pares ordenados: (A → B) significa que hay una conexión de A hacia B, pero no necesariamente de B hacia A.
- **Grafo no dirigido**
  - Las aristas son bidireccionales: si hay una arista entre A y B, se puede ir en ambos sentidos.
  - Se representan como pares no ordenados: {A, B}.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Grafo ponderado**

- A cada arista se le asigna un peso o valor numérico.
- El peso puede representar:
  - **Distancia** (en mapas)
  - **Coste** (en redes de transporte)
  - **Tiempo**, **capacidad**, etc.

- **Grafo conexo**

- Un grafo es conexo si existe al menos un camino entre cualquier par de vértices.
  - En los **grafos no dirigidos**, esto significa que no hay vértices aislados.
- En grafos dirigidos, se habla de:
  - **Fuertemente conexo**: hay camino en ambas direcciones entre todos los nodos.
  - **Débilmente conexo**: hay conexión si se ignora la dirección.

- **Grafo cíclico / acíclico**

  - **Cíclico**: contiene al menos un **camino cerrado** (un ciclo).
  - **Acíclico**: no contiene ciclos.
    - Un caso especial es el **Grafo Dirigido Acíclico (DAG – Directed Acyclic Graph)**, utilizado para representar flujos donde no puede haber bucles, como:

Representación de grafos

- **Matriz de adyacencia:**
  - Matriz cuadrada donde cada celda (i, j) indica si hay una arista entre los vértices i y j.
  - Ocupa **mucho espacio**, pero permite comprobar conexiones en **O(1)**.
- **Lista de adyacencia:**
  - Cada vértice tiene una lista con los nodos a los que está conectado.
  - **Más eficiente** en grafos dispersos (pocos enlaces).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.4 ESTRUCTURAS DE ACCESO DIRECTO

Las **estructuras de acceso directo** permiten localizar datos de forma eficiente utilizando una clave. Son especialmente útiles cuando se necesita realizar búsquedas rápidas o acceder a valores sin recorrer secuencias completas. Las principales estructuras de este tipo son las **tablas hash** y los **diccionarios**.

2.4.1 Tablas hash



Una **tabla hash** es una estructura de datos diseñada para realizar operaciones de **inserción, búsqueda y eliminación** de manera **muy eficiente**, idealmente en **tiempo constante O(1)** en el caso promedio.

La idea clave es utilizar una **función hash** para transformar una **clave** (por ejemplo, un número de DNI, un nombre o un identificador único) en un **índice** dentro de una tabla (array). Ese índice indica la **posición donde almacenar o buscar el valor asociado** a esa clave.

Funcionamiento básico:

1. Se define una **función hash** h(clave) que devuelve un número entero.
2. Este número se **mapea al tamaño de la tabla** usando el operador módulo: índice = h(clave) % tamaño_tabla.
3. El valor se **inserta** en la posición correspondiente del array.
4. Para **buscar o eliminar**, se aplica la misma función a la clave y se accede al índice correspondiente.

Este proceso es muy rápido, pero presenta un problema clave: las **colisiones**. Una **colisión** ocurre cuando **dos claves distintas generan el mismo índice**. Como cada posición debe almacenar un solo valor (en principio), se necesita una estrategia para manejar estas situaciones.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipos de resolución de colisiones:

1. **Hash con encadenamiento (chaining)**

- Cada posición de la tabla no almacena un solo valor, sino una **estructura secundaria** (normalmente una **lista enlazada** o una **lista dinámica**).
- Todos los elementos que tienen el mismo índice se almacenan **en esa lista**.

2. **Hash abierto o direccionamiento abierto (open addressing)**

- En lugar de usar listas, todos los elementos se almacenan **directamente en el array**.
- Si hay una colisión, se busca la **siguiente posición libre** según una estrategia determinada.
- Estrategias más comunes:
  - **Sondeo lineal (linear probing)**: se revisan las posiciones siguientes una a una (i+1, i+2...) hasta encontrar una vacía.
  - **Sondeo cuadrático (quadratic probing)**: el salto de búsqueda crece cuadráticamente ($i+1^2$, $i+2^2$...).
  - **Doble hash (double hashing)**: se aplica una segunda función hash para calcular el tamaño del salto.



- Tamaño de la tabla: conviene que sea un número primo para mejorar la dispersión de los índices.
- Factor de carga (load factor): relación entre el número de elementos y el tamaño de la tabla. Si es alto (> 0.7), conviene redimensionar la tabla para mantener la eficiencia.

2.4.2 Diccionarios

Un **diccionario** es una estructura de datos que almacena **pares clave-valor**, donde cada **clave** es **única** y se utiliza para acceder a su **valor asociado**.

Se puede considerar una **generalización de un array** en el que los índices no tienen que ser números consecutivos, sino **cualquier tipo de dato hashable** (números, cadenas, identificadores, etc.).

Funcionamiento:

- Para **almacenar** un dato: diccionario[clave] = valor
- Para **consultarlo**: valor = diccionario[clave]
- Para **modificarlo**: se asigna un nuevo valor a la misma clave.
- Para **eliminarlo**: del diccionario[clave]

Implementación:

Los diccionarios pueden implementarse usando diferentes estructuras internas:

1. **Basados en tablas hash**
   - La clave se transforma en un índice mediante una función hash.
   - La mayoría de los lenguajes modernos implementan los diccionarios con **tablas hash** debido a su **rapidez de acceso**.

2. **Basados en árboles balanceados**
   - Se utilizan estructuras como **árboles rojo-negro (Red-Black Tree)** o **árboles AVL**.
   - Las claves se mantienen **ordenadas**, lo que permite operaciones de **búsqueda por rango** o recorridos ordenados.

3. **Basados en tries (cuando las claves son cadenas)**
   - Utilizan un **árbol de prefijos** para almacenar cadenas eficientemente.
   - Ideales para **búsqueda por prefijo**, autocompletado y diccionarios de texto.

2.5 CONJUNTOS (SETS).

Un conjunto es una estructura de datos que almacena una colección de elementos únicos, sin un orden particular y sin permitir duplicados.

Características principales: no permite valores repetidos (insertar un elemento ya existente no modifica la estructura) y no garantiza un orden concreto (salvo variantes como el conjunto ordenado).

Operaciones habituales: add(elemento), remove(elemento), contains(elemento), y las operaciones de teoría de conjuntos (unión, intersección, diferencia y diferencia simétrica entre dos conjuntos).

Implementación: internamente suele basarse en tablas hash (hash set), con operaciones en tiempo medio O(1), o en árboles balanceados (tree set), con complejidad O(log n) manteniendo los elementos ordenados.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. ALGORITMOS

Un **algoritmo** es un conjunto finito y ordenado de instrucciones o pasos que, aplicados de forma sistemática, permiten resolver un problema o realizar una tarea.

Características de un algoritmo:

- **Finitud**: debe terminar después de un número determinado de pasos.
- **Definición precisa**: cada paso debe estar claramente definido.
- **Entrada**: puede tener cero o más datos de entrada.
- **Salida**: debe producir al menos un resultado.
- **Eficiencia**: se busca que el algoritmo utilice la menor cantidad posible de tiempo y recursos.

3.1 TIPOS DE ALGORITMOS

Un algoritmo puede clasificarse según el **problema que resuelve** o el **enfoque que utiliza**. En este apartado nos centramos en los algoritmos de **búsqueda** y **ordenación**, fundamentales para el tratamiento de estructuras de datos.

3.1.1 Algoritmos de búsqueda

Los algoritmos de búsqueda permiten **localizar un elemento** dentro de una estructura de datos

- **Búsqueda lineal (secuencial)**
  - Recorre los elementos **uno a uno** desde el principio hasta encontrar el valor buscado o llegar al final.
  - **No requiere** que los elementos estén ordenados.
  - Complejidad temporal:
    - **Peor caso**: O(n)
    - **Mejor caso**: O(1) (si el elemento está en la primera posición)
- **Búsqueda binaria**
  - Requiere que los datos estén **ordenados previamente**.

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Compara el valor buscado con el del elemento central, y divide el rango en mitades sucesivas.
- **Complejidad temporal. Peor caso: O(log n). Mejor caso: O(1) (si el elemento buscado coincide con el elemento central en la primera comparación)**



La **complejidad temporal** de un algoritmo es una medida que describe **el tiempo que tarda en ejecutarse** en función del **tamaño de la entrada** (normalmente representado como n).

No se mide en segundos, sino en **número de operaciones fundamentales** que el algoritmo realiza, ya que esto permite **comparar algoritmos independientemente del hardware o lenguaje de programación**.

Se utiliza para:

- Evaluar la **eficiencia de un algoritmo**.
- Comparar el rendimiento de diferentes soluciones para un mismo
- problema.
- Prever cómo escalará el algoritmo cuando n crezca mucho.
- **O(n)**: tiempo proporcional al tamaño de la entrada (lineal).
- **O(n^2)**: tiempo proporcional al cuadrado del tamaño de la entrada.
- **O(log n)**: tiempo proporcional al logaritmo del tamaño (muy eficiente).
- **O(1)**: tiempo constante, no depende del tamaño de la entrada.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.1.2 Algoritmos de ordenación

Los algoritmos de ordenación reordenan los elementos de una estructura siguiendo un criterio (normalmente de menor a mayor o alfabético). Se clasifican por su complejidad, estabilidad y estrategia.



- **Burbuja (Bubble Sort)**
    - Compara elementos adyacentes e intercambia si están desordenados.
    - Repite el proceso hasta que toda la lista está ordenada.
    - Complejidad temporal: **$O(n^2)$**

- **Inserción (Insertion Sort)**
    - Inserta cada elemento en su posición correcta dentro de la parte ya ordenada de la lista.
    - Muy eficiente para listas pequeñas o parcialmente ordenadas.
    - Complejidad temporal:
        - **Peor caso: $O(n^2)$**
        - **Mejor caso (lista ordenada): O(n)**

- **Selección (Selection Sort)**
    - Encuentra el menor elemento en cada pasada y lo coloca en su posición correspondiente.
    - No es estable, aunque existen versiones que sí lo son.
    - Complejidad temporal: **$O(n^2)$**

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Quicksort**
  - Algoritmo de **divide y vencerás**: selecciona un elemento como pivote, y divide el conjunto en menores y mayores, que se ordenan recursivamente.
  - Complejidad temporal:
    - Peor caso: **$O(n^2)$ (si el pivote es el mínimo o máximo)**
    - Promedio: **O(n log n)**

- **Mergesort**
  - También de tipo **divide y vencerás**. Divide el conjunto en mitades, las ordena y luego **fusiona** las sublistas.
  - Complejidad temporal: **O(n log n) en todos los casos**

En la práctica, la mayoría de lenguajes no implementan estos algoritmos de forma pura. Python (sorted()) y Java (Arrays.sort() para objetos) usan Timsort, un híbrido de Mergesort e Insertion Sort; C++ (std::sort) usa Introsort, un híbrido de Quicksort, Heapsort e Insertion Sort. Ambos evitan el peor caso de su algoritmo base garantizando O(n log n).

3.2 RECURSIVIDAD

La **recursividad** es una técnica de programación en la que una función **se llama a sí misma** para resolver **subproblemas más simples** del problema original.

Se basa en el principio de **dividir un problema en partes más pequeñas** de la misma naturaleza, resolviendo cada una de ellas de forma recursiva hasta alcanzar una solución final.

Estructura básica de una función recursiva:

Una función recursiva debe cumplir dos requisitos fundamentales:

1. **Caso base (condición de parada)**

   Es la condición que **detiene la recursión**. Sin caso base, la función entraría en una llamada infinita y provocaría un error (desbordamiento de pila).

2. **Llamada recursiva**

   Es la instrucción mediante la cual la función se **llama a sí misma** con una **versión más sencilla del problema** (por ejemplo, con un valor más pequeño).

© Reservad

Tipos de recursividad:

La recursividad puede clasificarse en función de **cómo y cuántas veces** se llama a la función dentro de su propia definición. Esta clasificación es importante para entender su **estructura interna**, su **eficiencia** y su **adecuación al problema** a resolver.

1. **Recursividad directa:** Ocurre cuando una función **se llama a sí misma de forma explícita** dentro de su propio cuerpo.

2. **Recursividad indirecta:** Se produce cuando una función A llama a otra función B, que a su vez vuelve a llamar a A (o a otra que termina llamando a A). Puede involucrar más de dos funciones.

3. **Recursividad lineal:** La función se llama a sí misma solo una vez por cada ejecución, y no realiza ninguna otra llamada recursiva adicional dentro de sí misma.

4. **Recursividad múltiple:** En este caso, la función se llama a sí misma más de una vez dentro de la misma ejecución. Esto genera un árbol de llamadas y puede crecer exponencialmente si no se gestiona bien.

5. **Recursividad de cola (tail recursion):** Es una forma especial de recursividad en la que **la llamada recursiva es la última operación que realiza la función** antes de retornar el resultado.

3.3 ALGORITMOS SOBRE ESTRUCTURAS DE DATOS

Los algoritmos sobre estructuras de datos permiten **insertar, eliminar, buscar y recorrer** elementos de manera eficiente. La elección de la estructura afecta directamente al **rendimiento** de estos algoritmos en tiempo y espacio.

3.3.1 Inserción y eliminación

Las operaciones de **inserción** y **eliminación** son fundamentales en estructuras como listas, pilas, colas, arrays y tablas hash. La eficiencia de estas operaciones depende directamente de la naturaleza de la estructura.

- **Inserción:** Consiste en añadir un nuevo elemento a una estructura de datos. Puede hacerse en diferentes posiciones.
    - Al principio
    - Al final
    - En una posición concreta
    - Según prioridad

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Eliminación:** Consiste en **quitar un elemento existente**. Puede realizarse de diferentes formas.
  - Posición
  - Valor
  - Regla (ej. LIFO, FIFO, prioridad)

3.3.2 Recorrido de árboles y grafos

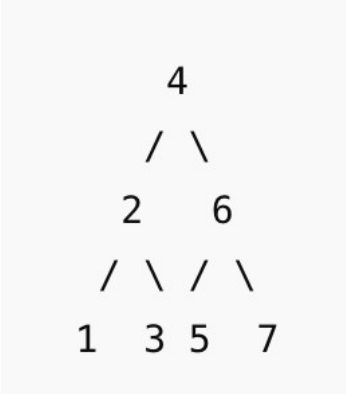
El **recorrido** es el proceso de visitar sistemáticamente todos los nodos de una estructura, normalmente para procesar, buscar o transformar información.

3.3.2.1 Recorrido de árboles

EXAMEN

Los árboles, especialmente los **árboles binarios**, se recorren habitualmente mediante tres estrategias clásicas, todas aplicables de forma **recursiva** o **iterativa.**

Considerando un árbol como el que se muestra en la imagen, se detalla en cada caso el resultado obtenido.



- **Inorden (in-order): izquierda → nodo → derecha**
  - En árboles binarios de búsqueda (BST), devuelve los elementos **ordenados**.
  - Requiere recorrer el subárbol izquierdo **antes** de procesar el nodo.
  - **Resultado recorrido árbol imagen: 1 → 2 → 3 → 4 → 5 → 6 → 7**

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Preorden (pre-order)**: nodo → izquierda → derecha
    - Útil para **copiar árboles** o **serializarlos**.
    - Procesa el nodo **antes** de visitar sus hijos.
    - **Resultado recorrido árbol imagen: 4 → 2 → 1 → 3 → 6 → 5 → 7**

- **Postorden (post-order)**: izquierda → derecha → nodo
    - Utilizado para **liberar memoria** (eliminar nodos tras sus hijos).
    - Se procesa el nodo **después** de sus hijos.
    - **Resultado recorrido árbol imagen: 1 → 3 → 2 → 5 → 7 → 6 → 4**

La complejidad temporal en todos los casos es O(n), siendo n el número total de nodos.

3.3.2.2 Recorrido de grafos

Los grafos presentan una topología más compleja que los árboles. Para recorrerlos, se utilizan dos algoritmos fundamentales:

- **DFS – Depth First Search (Búsqueda en profundidad)**
    - Explora tan lejos como sea posible a lo largo de cada rama antes de retroceder.
    - Implementación: mediante **pila** o **recursión**.
    - Complejidad: O(V + E), siendo V los vértices y E las aristas.

- **BFS – Breadth First Search (Búsqueda en anchura)**
    - Explora todos los nodos a un mismo nivel antes de descender al siguiente.
    - Implementación: mediante una **cola**.
    - Complejidad: O(V + E), al igual que DFS.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4. ORGANIZACIONES DE FICHEROS

Un **fichero** (o archivo) es una colección de datos almacenados en un medio secundario (disco duro, SSD, etc.), identificados por un nombre y utilizados por programas o sistemas para guardar información de manera persistente.

Tipos de ficheros:

- **Ficheros de texto**: contienen caracteres legibles por humanos. Ej.: .txt, .csv, .xml, .json.
- **Ficheros binarios**: almacenan datos en formato binario. Son más eficientes, pero no legibles directamente. Ej.: .exe, .dat, .bin.
- **Ficheros estructurados**: tienen un formato definido por registros y campos, como bases de datos o ficheros de configuración estructurados.

4.1 MODOS DE ACCESO A FICHEROS

El **modo de acceso a un fichero** define la forma en que los datos contenidos en él pueden ser **leídos, escritos o modificados**. Este modo determina cómo se localizan los registros dentro del archivo y tiene un gran impacto en el **rendimiento**, especialmente cuando se trabaja con grandes volúmenes de datos.

4.1.1 Acceso secuencial

El **acceso secuencial** es el modo más **simple, directo y tradicional** de leer y escribir datos en un fichero. En este modo, los **registros se recorren uno a uno** desde el principio hasta el final, en el **orden físico** en que fueron almacenados.

Este tipo de acceso se asemeja a **leer una cinta de casete o un archivo de vídeo**: no se puede saltar directamente a un punto concreto, sino que es necesario avanzar secuencialmente hasta llegar al registro deseado.

Características:

- La lectura y escritura se efectúan **en orden lineal**, sin saltos aleatorios.
- No se puede acceder directamente a un registro específico sin **recorrer todos los anteriores**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Es eficiente cuando se trabaja con **registros ordenados cronológicamente** o cuando se desea **procesar todos los datos sin excepción.**
- Generalmente se utiliza cuando los **registros tienen tamaño fijo** y una estructura homogénea.

4.1.2 Acceso directo o aleatorio (random access)

El **acceso directo o aleatorio** permite acceder **inmediatamente a un registro específico** dentro de un fichero, sin necesidad de recorrer los anteriores. Esto se consigue mediante una **fórmula matemática o función de localización**, basada en una **clave** o en la **posición lógica** del registro.

Este tipo de acceso es especialmente útil en sistemas que requieren **consultas rápidas** a datos individuales y se basa en la premisa de que **todos los registros tienen una ubicación predecible**, lo que generalmente implica que tengan **longitud fija**.

Características:

- Cada registro del fichero tiene una **posición fija** o puede calcularse a partir de una clave o número de registro.
- Se puede **saltar directamente** al registro deseado sin procesar los anteriores.
- El acceso se realiza usando una función del tipo:
  posición = dirección_base + (número_registro × tamaño_registro)
- Requiere que los registros estén organizados de forma uniforme (longitud fija) o que exista un **índice estructurado.**

4.1.3 Acceso indexado

El **acceso indexado** es una técnica que permite localizar registros de forma rápida mediante el uso de **estructuras de índices auxiliares**. Estas estructuras funcionan de manera similar al índice de un libro: permiten encontrar la posición exacta de un registro sin tener que recorrer el fichero completo.

Los **índices** suelen estar organizados como **tablas**, **árboles (como B o B+)** o **estructuras hash**, y contienen una **asociación entre claves** (como identificadores, nombres o códigos) y la **posición física** de los registros correspondientes dentro del archivo.

Este método **combina las ventajas del acceso secuencial y del acceso aleatorio**, lo que lo convierte en una opción muy versátil y eficiente.

Características:

- Los registros pueden mantenerse **ordenados por una o más claves** (clave primaria y secundarias).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- El fichero mantiene una o más **estructuras de índice**, que pueden ser internas (en el mismo archivo) o externas.
- La **búsqueda de un registro** se realiza primero en el índice, que lleva directamente a su posición.
- Es posible tener **índices secundarios** para búsquedas por distintos campos.
- Algunos sistemas permiten mantener el **índice en memoria** para acelerar aún más las búsquedas.

4.2 MÉTODOS DE ORGANIZACIÓN

Los **métodos de organización de ficheros** determinan cómo se **almacenan físicamente** los registros dentro de un archivo y cómo se **gestionan las operaciones** de acceso, inserción, modificación y eliminación.

4.2.1 Organización secuencial



La **organización secuencial** es el método más básico y tradicional de almacenamiento de registros en ficheros. En este enfoque, los datos se **guardan de forma consecutiva**, es decir, **uno tras otro** en el mismo orden en el que se generan o según una **clave primaria ordenada.**

Cuando se desea acceder a un registro específico, el sistema debe **leer el archivo desde el principio**, analizando secuencialmente cada entrada hasta encontrar el registro deseado. Esto convierte al método en **muy eficiente para procesamiento completo** del fichero, pero poco práctico para búsquedas puntuales.

Características:

- Los registros pueden ser de **tamaño fijo o variable**, aunque normalmente se utiliza longitud fija para facilitar el acceso.
- Puede mantenerse un **orden lógico** de los datos basado en una clave (alfabética, numérica, cronológica...).
- El fichero crece de forma lineal conforme se van añadiendo nuevos registros.
- El sistema de lectura **no permite saltos**: todos los registros anteriores al deseado deben procesarse.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo:

Supongamos un fichero con el siguiente contenido, para encontrar el registro de María (003), el sistema debe leer primero las líneas de Ana (001) y Luis (002). Este proceso es simple, pero **poco eficiente** si María se encuentra al final de un archivo con miles de registros.

```
001;Ana;Contabilidad

002;Luis;Informática

003;María;Finanzas
```

4.2.2 Organización secuencial indexada

La **organización secuencial indexada** es una mejora del modelo secuencial tradicional que incorpora una **estructura de índice adicional** para facilitar el acceso rápido a los registros. Aunque los datos continúan almacenándose de forma **ordenada secuencialmente** (por una clave primaria), el índice permite **localizar registros o rangos concretos sin recorrer todo el archivo**.

Este método combina las ventajas del **acceso secuencial** (orden lógico, recorrido eficiente) con las del **acceso directo** (búsqueda más rápida mediante índice), y es muy utilizado en sistemas de bases de datos, ficheros ordenados o gestión documental.

Características:

- Los registros están **ordenados secuencialmente** en el fichero principal, generalmente por una **clave primaria única**.
- Se añade un **índice auxiliar**, que actúa como una **tabla de referencias** que asocia claves con posiciones físicas dentro del archivo.
- El índice puede estar implementado como una **tabla simple**, un **árbol B o B+**, o estructuras más complejas si hay múltiples claves.
- Se pueden mantener **varios niveles de índices**:
  - **Índice primario**: sobre la clave principal del registro.
  - **Índices secundarios**: permiten búsquedas por otros campos (ej. nombre, departamento, fecha).

Ejemplo:

Supongamos un fichero con registros de empleados ordenados por número de empleado, y un índice como este, si se desea encontrar el registro de EMP-018, se localiza en el índice que está entre EMP-010 y EMP-020, y se **accede directamente al bloque correspondiente del archivo**, reduciendo drásticamente el tiempo de búsqueda respecto al acceso secuencial puro.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Índice primario:

```
Clave    →    Posición
EMP-001  →    0
EMP-010  →    900
EMP-020  →    1800
```

4.2.3 Organización directa o aleatoria

La **organización directa** (también llamada **organización aleatoria** o **random access**) permite acceder a cualquier registro del fichero **de forma inmediata**, calculando su **posición lógica** a partir de una **clave** mediante una **función determinista** (como una multiplicación, suma, o función módulo).

En lugar de recorrer el fichero secuencialmente o utilizar un índice auxiliar, el sistema **calcula directamente** la ubicación del registro en el fichero a partir de su clave. Esta técnica ofrece un **altísimo rendimiento en operaciones de acceso directo**, siempre que se utilice correctamente.

Características:

- Requiere una **función de transformación** que convierta una clave (por ejemplo, un número de cuenta) en una **posición física o lógica** dentro del archivo.
- La clave debe permitir un cálculo **rápido, único y consistente** de la ubicación del registro.
- Funciona mejor cuando los **registros tienen tamaño fijo**, lo que permite calcular su desplazamiento desde el inicio del fichero.
- No requiere recorrer ni indexar el archivo: el acceso es inmediato si la función es adecuada.
- Puede complementarse con **estructuras de gestión de colisiones**, si la clave calculada coincide con la de otro registro.

Ejemplo:

Supongamos que queremos acceder a la información del cliente con ID 458. Si el tamaño de cada registro es de **100 bytes**, la fórmula para calcular la posición en el fichero sería:

```
posición = 100 × (458 - 1) = 45.700 bytes
```

El sistema accede directamente a esa dirección y lee el registro en una única operación, sin necesidad de buscar, recorrer o consultar índices.

© Reservac

4.2.4 Organización por dispersión (hashing)

La **organización por dispersión**, también conocida como **hashing**, consiste en aplicar una **función hash** que transforma una **clave** (numérica o alfanumérica) en una **dirección lógica o física** dentro del fichero. Esta dirección indica el lugar donde debe almacenarse o buscarse el registro correspondiente a esa clave.

El objetivo es permitir un **acceso directo e inmediato** al dato sin necesidad de recorrer ni indexar el archivo. Sin embargo, como varias claves pueden generar la misma dirección (**colisiones**), es necesario implementar técnicas para resolver esos conflictos.

Características:

- El almacenamiento y búsqueda de datos se basan en el **cálculo de una función hash** que produce un índice o posición en el fichero.
- Las claves pueden ser de distintos tipos: **numéricas, alfabéticas o mixtas**.
- Es fundamental elegir una **buena función hash**, que distribuya uniformemente las claves para evitar demasiadas colisiones.
- Existen varias **estrategias de resolución de colisiones**, entre las que destacan:
  - **Encadenamiento**: cada posición de la tabla apunta a una lista enlazada con todos los registros que colisionan.
  - **Direccionamiento abierto**: en caso de colisión, se buscan otras posiciones disponibles en la tabla según una política (lineal, cuadrática, doble hashing...).

Ejemplo práctico

Supongamos que queremos almacenar información de usuarios identificados por su DNI. Aplicamos una función hash que convierte cada DNI en un número de posición dentro del fichero:

```
posición = hash("12345678A") → 145
```

Si esa posición ya está ocupada (colisión), se aplica una estrategia como **encadenamiento**, creando una lista de registros con la misma posición base, o **direccionamiento abierto**, buscando la siguiente posición libre según una fórmula.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. FORMATOS DE INFORMACIÓN Y FICHEROS

Los **formatos de información y ficheros** definen cómo se estructura y codifica la información dentro de un archivo. Cada formato tiene un propósito y características específicas relacionadas con su contenido, legibilidad, eficiencia y compatibilidad.

5.1 FORMATOS DE TEXTO

Son ficheros legibles por humanos y manipulables con editores de texto. Se usan ampliamente por su simplicidad y portabilidad.

- **TXT**: texto plano sin formato. Muy ligero, sin etiquetas ni estructura.
- **CSV (Comma Separated Values)**: formato estructurado donde los datos se organizan en filas y columnas separadas por comas (o punto y coma). Usado para exportación de hojas de cálculo.
- **JSON (JavaScript Object Notation)**: formato ligero y estructurado para intercambio de datos. Basado en pares clave-valor. Muy usado en APIs web.
- **XML (eXtensible Markup Language)**: formato jerárquico con etiquetas personalizadas. Muy flexible y autodescriptivo, pero más pesado que JSON.
- **YAML (YAML Ain't Markup Language)**: formato legible y estructurado similar a JSON, pero más conciso. Muy usado en configuraciones (ej. Docker, Ansible).

5.2 FORMATOS BINARIOS

Almacenan datos en formato no legible directamente. Son más eficientes y compactos, pero requieren software específico para su interpretación.

- **BIN**: contiene datos binarios sin estructura estándar. Usado en imágenes de disco, firmware, etc.
- **EXE**: ejecutable de Windows. Contiene instrucciones compiladas listas para ejecución.
- **DAT**: archivo genérico de datos binarios. Su contenido depende del programa que lo genera o utiliza.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5.3 COMPRESIÓN DE FICHEROS



Permiten reducir el tamaño de los archivos para ahorrar espacio y facilitar su transferencia. Pueden ser de compresión **con pérdida** o **sin pérdida**.

- **ZIP**: formato muy extendido con compresión sin pérdida. Admite múltiples archivos y carpetas.
- **RAR**: similar a ZIP, con mejor compresión en muchos casos. Requiere software específico (WinRAR).
- **GZIP**: compresor de archivos individuales, usado sobre todo en entornos Unix/Linux.
- **TAR**: empaqueta varios archivos sin comprimir. Se suele combinar con GZIP (.tar.gz) para compresión y agrupación.

5.4 FORMATOS DE FICHEROS COMUNES EN BASES DE DATOS

Son formatos diseñados para almacenar, exportar o intercambiar datos estructurados entre bases de datos.

- **MDB**: formato heredado (legacy) de bases de datos de Microsoft Access, sustituido como formato por defecto desde Access 2007 por ACCDB. Formato propietario, incluye datos, consultas, formularios, etc.; Access sigue permitiendo abrir y convertir archivos .mdb por compatibilidad.
- **DBF**: usado por dBASE y otros sistemas. Fichero plano con estructura de tabla.
- **SQL**: archivos de texto que contienen instrucciones SQL. Utilizados para exportar/importar bases de datos.
- **SQLite (archivo .sqlite o .db)**: base de datos ligera almacenada en un solo archivo binario. Muy usada en aplicaciones móviles o embebidas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

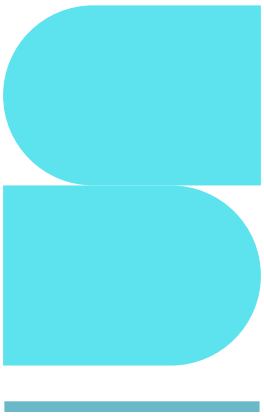
5.5 FORMATOS MULTIMEDIA Y DOCUMENTOS

Almacenan imágenes, vídeos, audio o documentos con formato. Son fundamentales en la gestión de contenidos digitales.

- **PDF (Portable Document Format)**: formato de documento multiplataforma que conserva el diseño original. Muy usado en administración y publicaciones.

- **DOCX**: documento de Microsoft Word basado en XML comprimido. Permite texto, estilos, tablas, imágenes...
- **JPG (JPEG)**: imagen con compresión con pérdida. Muy común por su buena relación calidad/ tamaño.
- **PNG**: imagen sin pérdida, admite transparencias. Ideal para gráficos web.
- **MP4**: formato contenedor de vídeo (y audio). Muy extendido en streaming y almacenamiento multimedia.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.